SKYLINE AIRLINES

Beginner-Friendly Code Explanation

A simple walkthrough of the C99 airline-management console project.

Guide section	What it covers
Program overview	How the app starts, loads files, and runs its menus
Data structures	The array, linked list, FIFO queue, stack, and binary search tree
Complete code reference	Each source file and all 37 C functions
Presentation help	Example booking flow and a short explanation to give in class

The admin ID is written in code/skyline.h. It is only a classroom demonstration check and is not secure authentication.

1. Quick start and program flow

Build and run: On Windows, install GCC (for example, MinGW), open a terminal in the project folder, and run run.bat. You can also double-click run.bat.

```
run.bat compiles C files -> program starts -> saved data loads -> menus appear
```

What happens when the program runs

• 1. main loads flights from flights.txt. If there are no saved flights, it creates and saves five sample flights.

• 2. The program loads confirmed bookings from bookings.txt and builds the flight-number search tree.

• 3. The user chooses the passenger menu or admin menu. Each menu calls a function for the requested task.

• 4. Booking either adds a confirmed booking and reduces available seats, or puts the passenger in the waiting queue.

• 5. Cancelling removes a booking and returns its seat. The first waiting passenger for that flight can be promoted.

• 6. The app saves the flights and bookings, then frees temporary linked structures before it exits.

2. Data structures in plain language

Structure	Plain meaning	Where it is used
Array	A fixed row of numbered slots.	Stores up to 50 flights.
Linked list	Separate nodes joined by next pointers.	Stores confirmed bookings.
FIFO queue	First In, First Out.	Waiting passengers join at the back; the earliest for a flight is chosen first.
Stack	Last In, First Out.	Recent actions are added to the top and shown newest-first.
Binary search tree	Smaller numbers go left; larger numbers go right.	Searches flight numbers for the admin BST option.

A pointer stores the location of another value in memory. In these structures, next, left, and right connect nodes. NULL means a link has no node.

3. Shared types and program state

code/skyline.h is the shared header. It sets size limits, defines the data structures, declares shared variables, and lists function prototypes. A prototype lets another file call a function by telling the compiler its name, inputs, and result.

Type or variable	What it means
Flight	A flight number, route, schedule, seat counts, and fare.
Booking	Confirmed booking details and a link to the next booking.
QueueNode / Queue	A waiting passenger node and the queue front/rear links.
StackNode	One action and a link to the next older action.
TreeNode	A flight number and left/right search links.
flights / flightCount	Flight array and number of filled entries.
nextBookingID	ID assigned to the next confirmed booking.
bookingHead	First confirmed-booking node.
waitingList	Front and rear of the waiting queue.
historyTop	Top of the recent-actions stack.
flightRoot	Root of the flight-number tree.

Program start

File: code/main.c

This file starts the app, restores saved information, shows the main menu, saves when exiting, and releases memory.

main()

```
int main(void)
```

Purpose: Loads saved flights and bookings, creates sample flights if needed, rebuilds the flight search tree, runs the main menu, saves data, and frees memory before exiting.

Inputs / result: Returns 0 when the program exits normally.

Shared state and memory

File: code/state.c

This file creates the one shared copy of each global variable and releases temporary data when the app ends.

freeAllMemory()

```
void freeAllMemory(void)
```

Purpose: Frees each booking node, waiting passenger node, and action-history node, then frees the flight search tree.

Inputs / result: No return value. The function uses the inputs shown below.

Keyboard input helpers

File: code/input.c

These functions make keyboard input reusable. Line input allows spaces in names and city names.

readLine()

```
void readLine(const char *prompt, char *buffer, int size)
```

Purpose: Prints the prompt, reads a line with spaces, removes the final newline, and clears characters that do not fit in the buffer.

Inputs / result: Writes text into buffer; no return value.

readInt()

```
int readInt(const char *prompt)
```

Purpose: Keeps asking until the input is one whole number. It rejects extra text such as 12abc. It returns 0 when input reaches end-of-file.

Inputs / result: Returns the entered integer; returns 0 at end-of-file.

readFloat()

```
float readFloat(const char *prompt)
```

Purpose: Keeps asking until the input is a valid, non-negative number. It returns 0 when input reaches end-of-file.

Inputs / result: Returns a non-negative number; returns 0 at end-of-file.

pauseScreen()

```
void pauseScreen(void)
```

Purpose: Waits for Enter before continuing to the next menu.

Inputs / result: No return value. The function uses the inputs shown below.

Flights and flight search

File: code/flights.c

Flights live in a fixed-size array. A binary search tree stores flight numbers for the admin tree-search option.

findFlightIndex()

```
int findFlightIndex(int flightNo)
```

Purpose: Checks each flight-array item and returns the matching array position. Returns -1 if the number is not present.

Inputs / result: Returns an array index or -1.

addSampleFlights()

```
void addSampleFlights(void)
```

Purpose: Copies five example flights into the flight array when there are no saved flights.

Inputs / result: No return value. The function uses the inputs shown below.

displayFlights()

```
void displayFlights(void)
```

Purpose: Prints every flight in the array with its route, schedule, available seats, and fare.

Inputs / result: No return value. The function uses the inputs shown below.

searchFlightByNumber()

```
void searchFlightByNumber(void)
```

Purpose: Asks for a flight number, finds it in the array, and prints its details or a not-found message.

Inputs / result: No return value. The function uses the inputs shown below.

searchFlightsByRoute()

```
void searchFlightsByRoute(void)
```

Purpose: Asks for the source and destination and prints flights where both city names match exactly. Matching is case-sensitive.

Inputs / result: No return value. The function uses the inputs shown below.

insertTree()

```
TreeNode *insertTree(TreeNode *root, int number)
```

Purpose: Inserts a flight number into its place in the binary search tree. Smaller values go left and larger values go right; duplicates are skipped.

Inputs / result: Returns the root of the updated tree or subtree.

searchTree()

```
TreeNode *searchTree(TreeNode *root, int number)
```

Purpose: Searches the tree by comparing the number and following one child link at a time. Returns the matching node or NULL.

Inputs / result: Returns a matching tree node or NULL.

freeTree()

```
void freeTree(TreeNode *root)
```

Purpose: Frees the left subtree, right subtree, then the current node, so all tree memory is released.

Inputs / result: No return value. The function uses the inputs shown below.

rebuildFlightTree()

```
void rebuildFlightTree(void)
```

Purpose: Frees the old tree and inserts the current flight numbers again. Called after flights are loaded or added.

Inputs / result: No return value. The function uses the inputs shown below.

searchFlightBST()

```
void searchFlightBST(void)
```

Purpose: Searches the tree for a number, then uses the flight array to print its route and fare.

Inputs / result: No return value. The function uses the inputs shown below.

addFlight()

```
void addFlight(void)
```

Purpose: Checks the maximum flight count and duplicate number, asks for flight details, adds the flight to the array, and rebuilds the search tree.

Inputs / result: No return value. The function uses the inputs shown below.

displayReport()

```
void displayReport(void)
```

Purpose: Totals flight seats and confirmed bookings, and estimates booking value by adding the fare for each confirmed booking.

Inputs / result: No return value. The function uses the inputs shown below.

Bookings, queue, and action stack

File: code/bookings.c

Confirmed bookings use a linked list, waiting passengers use a queue, and recent actions use a stack.

findBooking()

```
Booking *findBooking(int id)
```

Purpose: Walks from the first booking node until it finds the requested ID. Returns that node or NULL.

Inputs / result: No return value. The function uses the inputs shown below.

appendBooking()

```
int appendBooking(int id, const char *name, int age, int flightNo)
```

Purpose: Creates a booking node and attaches it to the end of the linked list. Returns 1 on success or 0 if memory allocation fails.

Inputs / result: Returns 1 on success or 0 if allocation fails.

displayAllBookings()

```
void displayAllBookings(void)
```

Purpose: Walks through and prints every confirmed booking in the linked list.

Inputs / result: No return value. The function uses the inputs shown below.

viewBooking()

```
void viewBooking(int passengerMode)
```

Purpose: Finds a booking ID and prints its e-ticket. Passenger mode also asks the passenger to confirm the booked name.

Inputs / result: No return value. The function uses the inputs shown below.

enqueueWaiting()

```
void enqueueWaiting(const char *name, int age, int flightNo)
```

Purpose: Creates a waiting-list node and adds it to the rear of the queue. The first node is both front and rear.

Inputs / result: No return value. The function uses the inputs shown below.

displayWaitingList()

```
void displayWaitingList(void)
```

Purpose: Prints passengers from the front of the queue to the rear, with their positions.

Inputs / result: No return value. The function uses the inputs shown below.

pushHistory()

```
void pushHistory(const char *action)
```

Purpose: Creates an action node and puts it at the top of the stack, where the newest action belongs.

Inputs / result: No return value. The function uses the inputs shown below.

displayActionHistory()

```
void displayActionHistory(void)
```

Purpose: Prints actions starting at the top of the stack, so the latest action appears first.

Inputs / result: No return value. The function uses the inputs shown below.

bookTicket()

```
void bookTicket(void)
```

Purpose: Checks the passenger name and age, asks which flight to book, and checks for a seat. It confirms the booking and reduces seats, or queues the passenger if the flight is full.

Inputs / result: No return value. The function uses the inputs shown below.

promoteWaitingPassenger()

```
void promoteWaitingPassenger(int flightNo)
```

Purpose: Finds the earliest waiting passenger for a flight with a free seat. It first creates their booking; only then does it remove them from the queue and use the seat.

Inputs / result: No return value. The function uses the inputs shown below.

cancelTicket()

```
void cancelTicket(int passengerMode)
```

Purpose: Finds a booking, checks the passenger name when needed, removes its node, returns the seat, records the action, and promotes a waiting passenger if possible.

Inputs / result: No return value. The function uses the inputs shown below.

Saving and loading

File: code/storage.c

This file reads and writes the text records that let flights and confirmed bookings survive a program restart.

saveFlights()

```
void saveFlights(void)
```

Purpose: Writes each flight as a pipe-separated line in flights.txt. Reports an error if the file cannot be opened.

Inputs / result: No return value. The function uses the inputs shown below.

loadFlights()

```
int loadFlights(void)
```

Purpose: Reads flights.txt into the array up to MAX_FLIGHTS. Returns 1 if at least one flight was read, otherwise 0.

Inputs / result: Returns 1 if one or more flights were read, otherwise 0.

saveBookings()

```
void saveBookings(void)
```

Purpose: Writes each confirmed booking in the linked list as one line in bookings.txt.

Inputs / result: No return value. The function uses the inputs shown below.

loadBookings()

```
void loadBookings(void)
```

Purpose: Reads bookings.txt, rebuilds the booking linked list, skips bookings whose flight is missing, and updates the next booking ID.

Inputs / result: No return value. The function uses the inputs shown below.

saveAll()

```
void saveAll(void)
```

Purpose: Calls saveFlights and saveBookings to save both persistent data files.

Inputs / result: No return value. The function uses the inputs shown below.

Passenger and admin menus

File: code/menus.c

This file prints the menus and calls the right feature function for the selected option.

passengerMenu()

```
void passengerMenu(void)
```

Purpose: Runs the passenger menu and calls the selected flight or booking feature. It saves data after a booking or cancellation.

Inputs / result: No return value. The function uses the inputs shown below.

ownerLogin()

```
int ownerLogin(void)
```

Purpose: Asks for the admin ID and allows up to three tries. Returns 1 when correct and 0 otherwise.

Inputs / result: Returns 1 for a successful login or 0 for failure.

ownerMenu()

```
void ownerMenu(void)
```

Purpose: Checks admin access, then runs the admin menu for managing flights and viewing bookings, waiting passengers, history, and reports.

Inputs / result: No return value. The function uses the inputs shown below.

4. Other project files and saved data

README.md

Project overview, features, data structures, build instructions, and a link to this code guide.

.gitignore

Tells Git to leave the generated build folder and local flights.txt and bookings.txt files out of the repository.

run.bat

Command	Purpose
@echo off	Hides the batch commands as they run.
setlocal	Keeps environment changes inside this script.
cd /d "%~dp0"	Changes to the project folder, including when it is on another drive.
if not exist build mkdir build	Creates the folder used for the executable.
gcc -std=c99 -Wall -Wextra -pedantic ...	Compiles the code as C99 and asks GCC to show warnings.
if errorlevel 1 exit /b %errorlevel%	Stops if compilation fails instead of trying to run a missing program.
build\skyline-airlines.exe	Starts the compiled application.

flights.txt and bookings.txt

The program reads and writes these pipe-separated text files in the project folder. A pipe character separates fields. The waiting queue and action history exist only in memory and are not saved.

```
Flight:  flightNo|source|destination|date|time|totalSeats|availableSeats|fare
Booking: bookingID|passengerName|age|flightNo
```

build/skyline-airlines.exe

Generated by run.bat. It is the compiled program, not source code, so Git ignores the build folder.

5. Example: booking and cancelling

• The passenger chooses a flight and enters a name and age.

• findFlightIndex checks the flight array for that flight number.

• If a seat exists, appendBooking creates a node and links it at the end of the booking list.

• The program lowers the flight seat count and gives the booking its ID.

• The menu saves the updated flight and booking records.

• For cancellation, cancelTicket finds and unlinks the booking node, then frees that node.

• The seat count increases. promoteWaitingPassenger checks the queue for the earliest passenger waiting for that flight.

• If a waiting passenger is found, the program creates their booking and the menu saves the new data.

6. A short project explanation

“Skyline Airlines is a C99 console application. Flights are stored in an array. Confirmed bookings use a linked list, full-flight passengers wait in a FIFO queue, recent actions use a stack, and an admin option searches flight numbers in a binary search tree. Flights and confirmed bookings are saved in text files. Separate files keep input, flights, bookings, storage, and menus easy to find.”

7. Limits to mention

• The array holds at most 50 flights.

• The admin ID is hard-coded in code/skyline.h. It is for a classroom demonstration, not secure authentication.

• The waiting queue and action stack reset when the program exits.

• The binary search tree is not self-balancing, so its shape depends on insertion order.

• Pipe-separated records cannot safely contain the pipe character inside a name or city.

• The data files are read and written in the current project folder.

Function coverage: 37 functions are explained across 7 C implementation files.